

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 7

Module 20: Database Connectivity

Module 21: Web Scraping

Module 22: API Handling

Continues from Parts 1-6 (Modules 1-19)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 20: DATABASE CONNECTIVITY	3
20.1 SQLite	3
20.2 MySQL Connection.....	3
20.3 CRUD Operations	4
Interview Questions — Module 20	4
MCQs — Module 20.....	5
Exercises — Module 20.....	5
MODULE 21: WEB SCRAPING	6
21.1 Requests.....	6
21.2 BeautifulSoup.....	6
Real Project — Extracting Headlines from a Page	7
Interview Questions — Module 21	8
MCQs — Module 21.....	8
Exercises — Module 21.....	8
MODULE 22: API HANDLING	9
22.1 REST API	9
22.2 JSON API.....	9
Real Project — Weather App Using a Public API	10
Interview Questions — Module 22	11
MCQs — Module 22.....	11
Exercises — Module 22.....	12
QUICK REVISION NOTES — MODULES 20-22	13
COMMON ERRORS AND FIXES — MODULES 20-22	14
CODING BEST PRACTICES — MODULES 20-22	15

MODULE 20: DATABASE CONNECTIVITY

Real applications need to store data permanently and query it efficiently — that's what databases are for. Python can connect to and control databases directly from code, letting you automate data storage and retrieval.

20.1 SQLite

SQLite is a lightweight, serverless database engine built into Python's standard library (via `sqlite3`) — no separate installation or server needed, making it ideal for learning and small applications.

```
import sqlite3

conn = sqlite3.connect("school.db") # creates the file if it doesn't exist
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS students (
        id INTEGER PRIMARY KEY,
        name TEXT,
        marks INTEGER
    )
""")
conn.commit()
```

Real-World Analogy

A database is like a well-organised filing cabinet. A table is one drawer (e.g., 'students'), and each row is one folder inside it — Python's `sqlite3` module lets your code open the cabinet, add folders, and search through them directly.

20.2 MySQL Connection

For larger, multi-user applications, a full database server like MySQL is used instead of SQLite. Python connects to it using an external library such as `mysql-connector-python` (installed with `pip`).

```
# pip install mysql-connector-python
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="yourpassword",
    database="school"
)
cursor = conn.cursor()
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)
```

Common Beginner Mistake

Forgetting to call `conn.commit()` after `INSERT/UPDATE/DELETE` means the changes are never actually saved to the database file — `SELECT` queries don't need it, but any change to data does.

20.3 CRUD Operations

CRUD stands for Create, Read, Update, Delete — the four basic operations every database-driven application performs.

```
# CREATE – insert a new record
cursor.execute("INSERT INTO students (name, marks) VALUES (?, ?)", ("Riya", 88))
conn.commit()

# READ – fetch records
cursor.execute("SELECT * FROM students")
print(cursor.fetchall())

# UPDATE – modify an existing record
cursor.execute("UPDATE students SET marks = ? WHERE name = ?", (95, "Riya"))
conn.commit()

# DELETE – remove a record
cursor.execute("DELETE FROM students WHERE name = ?", ("Riya",))
conn.commit()

conn.close()
```

SQL Injection Warning

Never build queries with an f-string or string concatenation of raw user input, e.g. `f"SELECT * FROM users WHERE name='{name}'"` — this is unsafe (SQL injection). Always use parameterised queries with placeholders (`?` for SQLite, `%s` for MySQL) as shown above.

Interview Questions — Module 20**Q1. What is the difference between SQLite and MySQL?**

Ans: SQLite is a lightweight, serverless, file-based database built into Python — ideal for small apps and learning. MySQL is a full client-server database engine designed for larger, multi-user, production applications, requiring a separate server installation.

Q2. What does the cursor object do in database connectivity?

Ans: It is used to execute SQL commands against the database and to fetch results back — think of it as the tool that actually 'moves through' and interacts with the data after the connection is established.

Q3. Why is `conn.commit()` necessary?

Ans: Changes made via `INSERT`, `UPDATE`, or `DELETE` are held in a transaction and are not permanently saved to the database until `commit()` is called — forgetting it means the changes are lost when the connection closes.

Q4. What does CRUD stand for?

Ans: Create, Read, Update, Delete — the four fundamental operations any application performs on stored data.

Q5. What is SQL injection, and how do parameterised queries prevent it?

Ans: SQL injection is an attack where malicious input is crafted to alter a SQL query's meaning, often by directly inserting user input into a query string. Parameterised queries (using placeholders like ? or %s) keep user input strictly as data, never as executable SQL, preventing this.

MCQs — Module 20

No.	Question	Answer	Options (A-D)
1	Which module provides built-in SQLite support in Python?	C	A) mysql B) db C) sqlite3 D) sqlengine
2	Which SQL keyword is used to add new data?	A	A) INSERT B) ADD C) CREATE D) NEW
3	Which method saves changes permanently to the database?	D	A) save() B) execute() C) write() D) commit()
4	CRUD stands for:	B	A) Create Read Use Delete B) Create Read Update Delete C) Copy Read Update Delete D) Create Remove Update Data
5	Why should you use parameterised queries?	C	A) They run faster always B) They format output C) They prevent SQL injection D) They are required by Python

Exercises — Module 20

1. Create an SQLite database with a 'books' table (id, title, author, price) and insert 5 records.
2. Write a program to update the price of a book by its title.
3. Write a program to delete a book record by id.
4. Write a program to fetch and print all books with price greater than 300.

MODULE 21: WEB SCRAPING

Web scraping is the automated extraction of data from websites — useful for collecting prices, news, listings, or research data that isn't available through an official API. It combines two libraries: requests (to download a page) and BeautifulSoup (to parse and search its HTML).

Ethics and Legality

Always check a website's robots.txt and Terms of Service before scraping it. Scrape responsibly: don't overload a server with rapid requests, and never scrape data you don't have the right to use.

21.1 Requests

The requests library fetches the raw HTML content of a web page, just like a browser would.

```
# pip install requests
import requests

response =
requests.get("https://example.com")
print(response.status_code)  # 200
means success
print(response.text[:200])  #
first 200 characters of the HTML
```

Output:

```
200
<!doctype html><html>...</html>
```

Status Code	Meaning	Example	Result
200	OK — request succeeded	response.status_code	200
404	Not Found — page doesn't exist	response.status_code	404
403	Forbidden — access denied	response.status_code	403
500	Server Error	response.status_code	500

21.2 BeautifulSoup

BeautifulSoup parses raw HTML into a navigable structure, letting you search for specific tags, classes, or IDs to extract exactly the data you need.

```
# pip install beautifulsoup4
from bs4 import BeautifulSoup
import requests

html = "<html><body><h1>Latest News</h1><p class=\"item\">Story 1</p><p
class=\"item\">Story 2</p></body></html>"
soup = BeautifulSoup(html, "html.parser")

print(soup.h1.text)                                # get text of the h1 tag
items = soup.find_all("p", class_="item")          # find all matching <p> tags
for item in items:
    print(item.text)
```

Output:

```
Latest News
Story 1
Story 2
```

Real Project — Extracting Headlines from a Page

```
import requests
from bs4 import BeautifulSoup

url = "https://example-news-site.com"
response = requests.get(url)

if response.status_code == 200:
    soup = BeautifulSoup(response.text, "html.parser")
    headlines = soup.find_all("h2", class_="headline")
    for h in headlines:
        print(h.text.strip())
else:
    print("Failed to fetch page:", response.status_code)
```

Line	Explanation
5	Send a GET request to download the page's raw HTML.
7	Always check status_code before parsing — scraping a failed response wastes effort and can crash your program.
8-10	Parse the HTML and search for every <h2> tag with class 'headline', then print each one's text.

Interview Questions — Module 21

Q1. What is the difference between the requests and BeautifulSoup libraries?

Ans: requests handles the network side — sending an HTTP request and downloading the raw HTML/response from a URL. BeautifulSoup handles the parsing side — turning that raw HTML into a structured, searchable object to extract specific data from.

Q2. Why should you check response.status_code before parsing a page?

Ans: A non-200 status code (like 404 or 403) means the request did not succeed as expected, so parsing that response would fail or return no useful data — checking first avoids wasted work and confusing errors.

Q3. What is the difference between find() and find_all() in BeautifulSoup?

Ans: find() returns only the FIRST matching element (or None if not found). find_all() returns a list of ALL matching elements on the page.

Q4. Why is it important to respect robots.txt and a site's Terms of Service when scraping?

Ans: robots.txt indicates which parts of a site the owner permits automated tools to access; ignoring it (or the site's terms) can be against the site's policy, may be illegal depending on jurisdiction and use case, and can get your IP blocked.

Q5. What could cause a web scraping script to suddenly stop working?

Ans: Websites frequently change their HTML structure/class names, which breaks scrapers relying on specific tags; sites may also add anti-bot protections, requiring updates to the scraping logic (or making automated scraping infeasible).

MCQs — Module 21

No.	Question	Answer	Options (A-D)
1	Which library downloads a web page's HTML?	B	A) bs4 B) requests C) sqlite3 D) json
2	Which library parses and searches HTML content?	C	A) requests B) urllib C) BeautifulSoup D) csv
3	A status code of 200 means:	A	A) Success B) Not found C) Server error D) Forbidden
4	Which method returns ALL matching tags?	D	A) find() B) get() C) search() D) find_all()
5	Before scraping a site, you should check:	C	A) Its CSS colours B) Its font C) robots.txt / Terms of Service D) Its logo

Exercises — Module 21

1. Write a program to fetch a webpage and print its HTTP status code.
2. Write a program to extract all links (<a> tags) from a sample HTML page.
3. Write a program to extract all paragraph text from a sample HTML page.
4. Write a program to count how many images (tags) are present on a sample page.

MODULE 22: API HANDLING

An API (Application Programming Interface) lets two programs communicate — most commonly, your Python script requesting data from a remote web service. This is the standard, official way to get data (weather, currency rates, social media info) instead of scraping.

22.1 REST API

A REST API exposes data through URLs (endpoints), typically returning JSON, and is accessed using standard HTTP methods:

HTTP Method	Purpose	Example	Result
GET	Retrieve data	<code>requests.get(url)</code>	reads data
POST	Create new data	<code>requests.post(url, data=...)</code>	creates a record
PUT	Update existing data	<code>requests.put(url, data=...)</code>	updates a record
DELETE	Remove data	<code>requests.delete(url)</code>	deletes a record

```
import requests

response =
requests.get("https://api.github.com/users/octocat")
print(response.status_code)
data = response.json()      # parse the JSON
                             response into a Python dict
print(data["login"])
print(data["public_repos"])
```

Output:

```
200
octocat
8
```

22.2 JSON API

Most REST APIs return data in JSON format. Python's requests library can parse this directly into a dictionary/list using `.json()` — no need to manually import the json module for this step.

```
import requests

url = "https://api.exchangerate-api.com/v4/latest/USD"
response = requests.get(url)

if response.status_code == 200:
    rates = response.json()
    print("1 USD =", rates["rates"]["INR"], "INR")
else:
    print("API request failed:", response.status_code)
```

Output:

```
1 USD = 83.12 INR
```

Real Project — Weather App Using a Public API

```
import requests

API_KEY = "your_api_key_here"
city = "Mumbai"
url =
f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"

response = requests.get(url)
if response.status_code == 200:
    data = response.json()
    temp = data["main"]["temp"]
    desc = data["weather"][0]["description"]
    print(f"{city}: {temp}°C, {desc}")
else:
    print("Error fetching weather:", response.status_code)
```

Line	Explanation
3-5	Build the request URL, inserting the API key and target city using an f-string.
7	Send the GET request to the weather API.

8-11	If successful, parse the JSON response and pull out just the fields we need — temperature and description.
12-13	Handle failure gracefully instead of crashing on a bad response.
Output: Mumbai: 31.5°C, clear sky	
Common Beginner Mistake Hardcoding API keys directly in shared or public code (like on GitHub) is a security risk — anyone can misuse your key/quota. Store secrets in environment variables or a separate untracked config file instead.	

Interview Questions — Module 22

Q1. What is an API, in simple terms?

Ans: An API is a defined way for one piece of software to request services or data from another — like a waiter taking your order (request) to the kitchen (server) and bringing back your food (response), without you needing to know how the kitchen works internally.

Q2. What is the difference between GET and POST HTTP methods?

Ans: GET retrieves data from the server without changing anything (e.g., fetching weather data). POST sends new data to the server to create something (e.g., submitting a form or creating a new record).

Q3. How do you convert an API's JSON response into a Python object?

Ans: Using `response.json()` on a requests Response object, which parses the JSON text directly into a Python dictionary or list.

Q4. Why should you always check the status code before using API response data?

Ans: A failed request (e.g., 404, 401, 500) won't contain the expected data structure, so trying to parse/access fields from it directly would raise errors — checking first lets you handle failures gracefully.

Q5. Why shouldn't you hardcode API keys directly in your source code?

Ans: Hardcoded keys can be accidentally exposed (e.g., if the code is pushed to a public repository), letting others misuse your account, quota, or billing. Storing keys in environment variables or config files kept out of version control is safer.

MCQs — Module 22

No.	Question	Answer	Options (A-D)
-----	----------	--------	---------------

No.	Question	Answer	Options (A-D)
1	Which HTTP method retrieves data without changing it?	A	A) GET B) POST C) PUT D) DELETE
2	Which method converts a requests response into a Python dict?	C	A) .dict() B) .parse() C) .json() D) .load()
3	Most REST APIs return data in which format?	B	A) CSV B) JSON C) XML only D) Plain text only
4	Which HTTP method is used to create a new resource?	D	A) GET B) DELETE C) PUT D) POST
5	Storing API keys directly in public source code is:	A	A) A security risk B) Best practice C) Required D) Faster

Exercises — Module 22

1. Use a free public API (e.g., a joke or quote API) to fetch and print a random result.
2. Write a program to fetch and display current exchange rates for 3 currencies.
3. Write a program that fetches GitHub user info and prints their name, bio, and follower count.
4. Write a program that handles a failed API call gracefully with a clear error message.

QUICK REVISION NOTES — MODULES 20-22

- `sqlite3` (built-in) is ideal for small/local apps; `mysql-connector-python` connects to a full MySQL server for larger applications.
- CRUD = Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE). Always call `conn.commit()` after changes.
- Always use parameterised queries (`?, %s`) — never insert raw user input directly into an SQL string.
- Web scraping = `requests` (download HTML) + `BeautifulSoup` (parse/search HTML). Always check `robots.txt` and `status_code` first.
- APIs are the official way to exchange data between programs; REST APIs use GET/POST/PUT/DELETE and typically return JSON.
- `response.json()` converts an API's JSON response directly into a Python dict/list.
- Never hardcode API keys or passwords in shared/public code — use environment variables or config files.

COMMON ERRORS AND FIXES — MODULES 20-22

Error	Cause	Fix
sqlite3.OperationalError: no such table	Table was never created, or wrong DB file	Run the CREATE TABLE statement first
Changes not saved to database	Missing conn.commit()	Call commit() after INSERT/UPDATE/DELETE
ConnectionError / Timeout	No internet access or wrong URL	Check the URL and network connection
AttributeError: NoneType has no attribute 'text'	BeautifulSoup's find() found nothing (page structure changed)	Check the tag/class exists; verify by printing the raw HTML first
JSONDecodeError on response.json()	The response wasn't actually JSON (e.g., an HTML error page)	Check status_code and response.text before calling .json()

CODING BEST PRACTICES — MODULES 20-22

- Always use parameterised SQL queries — never format raw user input directly into a query string.
- Close database connections (`conn.close()`) when done, or use a 'with' context manager where supported.
- Always check `response.status_code` before parsing scraped HTML or API JSON responses.
- Respect rate limits and robots.txt when scraping; add delays between requests to avoid overloading servers.
- Keep API keys and passwords out of source code — use environment variables or a .env file excluded from version control.
- Wrap network calls (requests, database queries) in try-except to handle failures gracefully instead of crashing.